

A Secure and Efficient Data Deduplication Scheme with Dynamic Ownership Management in Cloud Computing

Xuwei Ma, Wenyuan Yang, Yuesheng Zhu, Zhiqiang Bai

School of Electronic and Computer Engineering, Peking University

mxw971201@stu.pku.edu.cn, wyyang@pku.edu.cn, zhuys@pku.edu.cn, baizq@pku.edu.cn

Abstract—Encrypted data deduplication is an important technique for eliminating duplicate copies of repeating data, and has been widely used in cloud storage to save storage space and network bandwidth. Recently, several deduplication schemes solving the privacy-preserving problem of dynamic ownership management have been proposed. However, these schemes suffer from low efficiency when the cloud user joining and revocation frequently go on, especially in the absence of a trusted third party. In this paper, we propose a novel server-side deduplication scheme for encrypted data in a hybrid cloud architecture, where a public cloud (Pub-CSP) manages the storage and a private cloud (Pri-CSP) plays a role as the data owner to perform deduplication and dynamic ownership management. Further, to mitigate the communication overhead we adopt a pre-verified accessing control approach to prevent the unauthorized cloud users from downloading data and use an initial uploader check mechanism to ensure only the first uploader needs to perform encryption. Our security analysis and performance evaluation demonstrate that our proposed scheme has better performance in terms of security, effectiveness, and practicability compared with other schemes.

Index Terms—Data Deduplication, Cloud Computing, Access Control, Storage Management, hybrid cloud

I. INTRODUCTION

Nowadays, with the rapid growth of data volumes, there are urged demands for secure places where private data can be safely stored. Outsourcing the big data to the cloud is an efficient way to solve this problem [1], [2]. Despite all the advantages of cloud computing [3], duplicated data storage could greatly waste networking resources and make data management complicated [4]. Duplicate data is generated when the multiple data users upload the same data encrypted with different keys to the cloud.

The deduplication [5] is a process that identifies the same data by data similarity, which allows the cloud storage provider save the storage by storing only a single copy of the data owned by multiple owners. However, the existing schemes related to deduplication still incur some problems about dynamic ownership management and access control.

First of all, to protect privacy, many users encrypt data before uploading it to the cloud storage. However, since the encryption key is randomly generated, the same data encrypted with different keys will produce different ciphertext, which will hinder deduplication. To solve this problem, some deduplication schemes propose that the owners of the same

file share the same encryption key [6]–[14]. However, most of them do not consider the dynamic changes in the ownership of outsourced data that happen frequently in a practical cloud storage service [15]. When the authorized users request the cloud storage provider to delete or modify the data, they should be revoked from the valid ownership list and prevented from accessing the data anymore.

Second, to address dynamic ownership management, many schemes [4], [15], [32]–[34] were proposed by using either trusted third party or semi-trusted third party to do proxy re-encryption work, such as Authority Party (*AP*) or The Public Cloud Provider (*Pub-CSP*). On the one hand, it is difficult to implement a trusted third party in real-world applications [35]. On the other hand, some schemes can not resist collusion attacks when the third party colludes with unauthorized users.

In this paper, we propose an novel scheme, which aims at efficiently solving the problem of deduplication with frequent cloud user revocation and new cloud user joining in cloud computing. In particular, different from existing data deduplication methods, which employ either trusted/semi-trusted third party to do proxy re-encryption work, our proposed scheme designs a hybrid cloud architecture, which includes a public cloud and further introduces a private cloud. In implementations, the introduced private cloud in our scheme is involved as a data owner and a proxy at the same time to 1) control access to outsourced data by performing re-encryption techniques and 2) manage the dynamic ownership when real data owner is offline or revoke his/her ownership. In this way, the data owners can outsource their data to the public cloud while the data deduplication is managed by the private cloud.

Furthermore, we propose to enhance our scheme in terms of efficiency by 1) ensuring that the data owner performs encryption only when he/she is the initial uploader; 2) presenting an access control technique to verify the validity of the cloud users before they download data; and 3) requiring the public cloud server can send ciphertext to the cloud user only when the cloud users are in the ownership list. Therefore, the abundant communication cost will be reduced. We evaluate the performance of our scheme through security analysis, comparison with existing work, and implementation-based performance evaluation.

Overall, our contributions are as follows:

- We propose a novel scheme, which introduces a hybrid

cloud architecture including a public cloud and a private cloud, to efficiently solve the problem of deduplication with frequent cloud user revocation and new cloud user joining in cloud computing.

- We further propose to improve our scheme in efficiency. The extensive experiments prove the security, effectiveness, and efficiency of our method, which can significantly outperform previous data deduplication methods.

II. RELATED WORK

A. Deduplication without Dynamic Ownership Management

To solve deduplication problem, convergent encryption [36] was proposed, in which a data user got the key of data F by computing its hash code $K = H(F)$. Since data F will be encrypted with K , whoever holds the same data can produce the exact same encrypted data.

Later, Bellare et al. [7] brought a new idea of a server-aided encryption scheme (DupLESS) for data deduplication. In DupLESS, an independent key server generates the key, which suffers from large computation time in block-level deduplication since it takes long time to generate keys. Liu et al. [37] proposed a secure scheme that supported client-side encryption by applying a password authenticated key exchange protocol. However it requests that the data owner is always online for data ownership check and deduplication, which is unpractical.

Cui et al. [12] proposed a deduplication scheme by using attribute-based access control technique under hybrid cloud environment. Recently, Vengala et al. [38] proposed a secured effective data transmission algorithm (CP-ABE-ECC) for augmenting the Security Level. The drawback of these schemes is not considering the dynamic ownership management among the data users.

B. Deduplication with Dynamic Ownership Management

To address dynamic ownership management, many schemes were proposed by using either a trusted third party or semi-trusted third party to do proxy re-encryption work, such as AP or Pub-CSP. Wen et al. [32] constructed a convergent key sharing scheme. But this work requires the data user to encrypt/decrypt convergent keys and recover them from secret shares, which is unrealistic since the data user's computation power is limited. Premkamal et al. [34] proposed an enhanced scheme with attribute-based access control by using the group key with the help of trusted entities. Later, an enhanced scheme proposed by Yan et al. [4] provided a heterogeneous data storage management. However, it is not practical for the owner to be always online and send personalized keys to the holders. Beside, once the owner is offline, the access control will be entirely dependent on AP.

Hur et al. [15] proposed a novel server-side deduplication scheme that assumed the total number of data users was fixed and known to the cloud server. However, it is difficult to determine the number of universe users before data uploading is performed. Later, Yuan et al. [33] proposed the improved deduplication scheme based on [15]. However, they do not

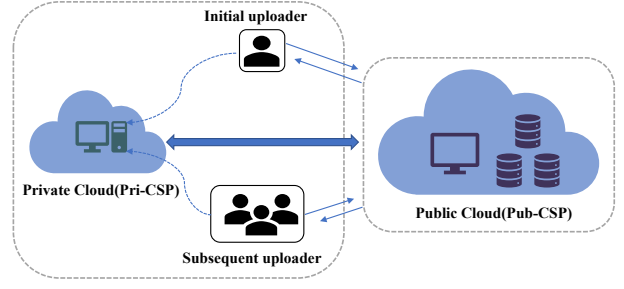


Fig. 1. Architecture of a data deduplication system.

verify that the holders hold the entire original file instead of only having the tag T , the fake ciphertext and ID .

Although these schemes address dynamic ownership management, there still are some security flaws. Since most of the schemes used either trusted/semi-trusted third party to do proxy re-encryption work, some data owners do not like to authorize a third party like AP to control their data. Besides, taking scheme [4] as an example, once the malicious user conspires with the third party (such as a proxy server), it will happen that unauthorized users u who only has access to file A can also have invalid access to file B . This is the problem we intend to solve in this study.

III. SYSTEM MODEL

In this section, we describe the data deduplication system and define the adversary model. We only consider the file-level deduplication in this paper.

A. Hybrid Architecture for Secure Deduplication

Figure 1 shows the architecture of the data deduplication system, which consists of three entities stated as follows.

- **Data users (DU).** A user is an entity that wants to outsource data to Pub-CSP and access data later. In the authorized deduplication system, each user is issued a pair of secret key and public key (sk_u, pk_u) about Proxy Re-Encryption (PRE) and (SK_u, PK_u) for signature. Each file is protected with the symmetric encryption key and re-encryption keys generated by PRE to realize the authorized deduplication. Moreover, if a data user is the first one to upload file F_A , she/he will be defined as the owner $u_{1,A}$; if the file F_A already exist, the user will be defined as the holder $u_{i,A}$.

- **Public Cloud (Pub-CSP).** This is an entity that provides a data storage service in the public cloud. In this paper, we assume that Pub-CSP is always online and has abundant storage capacity and computation power.

- **Private Cloud (Pri-CSP).** This is a new entity introduced for facilitating users' secure usage of cloud service. Specifically, Pri-CSP can provide an execution environment and infrastructure for data users as an interface between the user and Pub-CSP, since the computing resources of the data user (DU) are limited and Pub-CSP is not fully trusted in practice. Pri-CSP maintains an ownership list for storing data, which is composed of a hash code set for the stored data, identities, and re-encrypted keys belonging to the owners.

In this system, We assume that the infrastructure of Pri-CSP is exclusively used by a single organization consisting of multiple users. It is owned, managed, and operated by the organization itself. Therefore, Pri-CSP can be trusted by all entities. Pub-CSP will strictly follow system design and protocols strictly, while they may still be curious about the raw data of data users. Moreover, We assume that Pri-CSP and DU would never collude with Pub-CSP due to different business interests.

B. Security Requirements

We follow [15] to ensure the following security requirements: Data privacy, Data consistency, Ownership verification, Ownership revocation, Collusion resistance.

- **Data privacy.** Protecting plaintext from the Pub-CSP server and unauthorized users.
- **Data consistency.** Ensuring tag consistency against any poison attacks. The ciphertext modification can be verified by the authorized owners .
- **Ownership verification.** Unauthorized users who cannot prove ownerships should not be able to obtain the ciphertext or the re-encrypted key stored in the cloud storage.
- **Ownership revocation.** Authorized users who request the cloud storage provider to delete or modify the data F should be prevented from accessing the data F anymore.
- **Collusion resistance.** Unauthorized users who do not have valid ownership of data in the cloud storage should not be able to decrypt ciphertext even if they collide with each other or collide with Pub-CSP.

IV. PRELIMINARIES AND DEFINITION

A. Proxy Re-Encryption (PRE)

Proxy re-encryption is an asymmetric encryption technique initially proposed by Blaze et al. [39] in which an entity A (delegator) can delegate the decryption rights to another entity B (delegatee) through a proxy server. In the PRE technique, we have three main actors:

- **Delegator:** data owner who delegates decryption rights to another user (i.e., delegatee);
- **Delegatee:** data user who receives decryption rights to access the encrypted data;
- **Proxy:** performs a re-encryption using the delegation key (generated by the delegator) and the ciphertext to allow the delegatee to access the encrypted data;

Initially, data F is encrypted using the delegator's public key, $C_A = \text{En}(pk_A, F)$, and stored in a database. If delegatee B needs to decrypt the data, he must initially request decryption rights for the delegator informing his public key pk_B . If the delegator agrees, it will produce a delegation key by $rk_{A \rightarrow B} = \text{RG}(sk_A, pk_B)$ and send it to the proxy.

For delegatee B to be able to decrypt the data, the proxy server must use the delegation key $rk_{A \rightarrow B}$ to re-encrypt C_A , that is, $C_B = \text{ReEn}(rk_{A \rightarrow B}, C_A)$. After re-encryption, the delegatee can use his private key sk_B and decrypt the message. At all stages, only the public key is shared between the participants. From the proxy's point of view, it does not learn

TABLE I
NOTATION.

Notations	Description
F_{id}/F	The duplicated data F
u_{id}/u	The users of cloud service
K/K'	The symmetric key of F / Updated key
EK/EK'	The encrypted K/K'
REK/REK'	The encrypted EK/EK' by PRE
sk_{u_0}	The secret key of the Pri-CSP about PRE
pk_{u_0}	The public key of the Pri-CSP about PRE
sk_u	The secret key of u about PRE
pk_u	The public key of u about PRE
CT/CT'	The encrypted data F / Renewed ciphertext
$H(*)$	The hash function
$HC(F)$	The hash code set of data F

or try to decrypt confidential information. It receives encrypted information C_A and sends other encrypted information C_B .

In our scheme, Pri-CSP plays a role as **Delegator** and **Proxy** at the same time to reduce client-side computation overhead. Table 1 summarizes the notations used in this paper.

B. Definition

In this section, we define a secure deduplication framework for encrypted data with dynamic ownership management. The details of algorithms used in our scheme will be shown as follows.

1) System Setup

- **KeyGen1(u_{id}).** Based on the system parameters, user u generates its own key pair (sk_u, pk_u) about PRE and (SK_u, PK_u) for signature by inputting the unique u_{id} . Meanwhile, Pri-CSP gets (sk_{u_0}, pk_{u_0}) by having $u_{0,id}$ as owner u_0 .

2) Data Encryption and Decryption

- **Encrypt(K, F).** For plaintext data F , data owner u_1 encrypts it with the symmetric key K to get ciphertext CT .
- **Decrypt(K, CT).** Data holder u_i decrypts CT with key K and outputs F .

3) Key Control based on PRE Operated by Pri-CSP

- **KeyGen2(Λ)** takes a security parameter as input, and outputs a random symmetric key K .
- **En(pk_{u_0}, K)** takes pk_{u_0} and the symmetric key K as input and outputs an encrypted key EK .
- **RG(sk_{u_0}, pk_{u_i})** outputs re-encryption key $rk_{u_0 \rightarrow u_i}$ by taking sk_{u_0} and pk_{u_i} as input.
- **ReEn($rk_{u_0 \rightarrow u_i}, EK$)** takes input $rk_{u_0 \rightarrow u_i}$ and EK , and outputs $REK_{u_i} = \text{En}(pk_{u_i}, K)$ that can be decrypted with sk_{u_i} by $\text{De}(sk_{u_i}, REK_{u_i})$.

V. PROPOSED DEDUPLICATION SCHEME

A. Overview

We propose a secure deduplication scheme for encrypted data with dynamic ownership management. Since Pri-CSP plays a role as delegator and proxy at the same time, the deduplication is controlled by owner u_0 (Pri-CSP) to have less bandwidth and computational overhead for clients. Moreover, the dynamic ownership can be managed by owner u_0 or owner u_1 since different users have different needs.

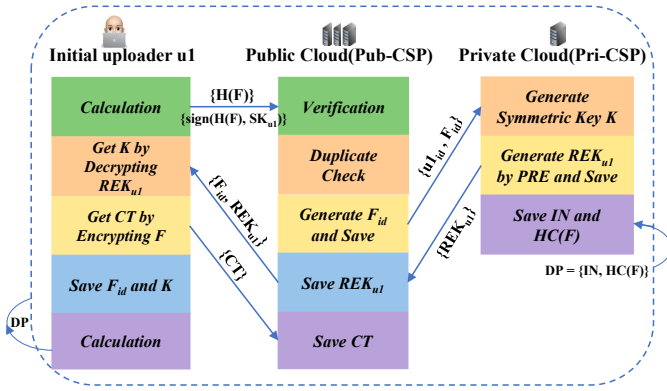


Fig. 2. Procedure of initial upload by owner u_1 .

B. Data Deduplication

Figure 2 shows the procedure of initial upload by owner u_1 . Figure 3 shows the procedure of deduplication when subsequent uploaders u_2 uploads the same data with u_1 . We assume that Pri-CSP plays a role as owner u_0 to control deduplication for owner u_1 .

Step 1 - Key Generation: After system parameter generation, each DU asks $KeyGen1(u_{id})$ to generate key pair (sk_u, pk_u) for PRE and (SK_u, PK_u) for signature. Meanwhile, Pri-CSP gets (sk_{u_0}, pk_{u_0}) by $KeyGen1(u_0)$.

Step 2 - Duplication Check: DU u_1 stores data F at Pub-CSP. He/She calculates $H(F)$, signs it with SK_{u_1} and sends data package $dp = \{H(F), sign(H(F), SK_{u_1})\}$ to Pub-CSP. Pub-CSP checks if the same data has been stored already after verifying the signature. If the check is positive, go to Step 5. Otherwise, go to Step 3 to request key for encryption.

Step 3 - Data Storage: When Pub-CSP defines u_1 is the first uploader of data F , it generates a unique F_{id} for F and contacts Pri-CSP to get key for encryption. Pri-CSP generates a random symmetric key K , and encryps it with pk_{u_0} to get EK , then applies PRE to EK to get cipherkey REK_{u_1} for DU u_1 . Pub-CSP sends (F_{id}, REK_{u_1}) to u_1 after receiving it from Pri-CSP. User u_1 decrypts REK_{u_1} by $De(sk_{u_1}, REK_{u_1})$ with his/her own private key to get K , and then u_1 encrypts data F by $Encrypt(K, F)$. In addition, user u_1 randomly selects a number of indexes: $IN = In_1, In_2, \dots, In_n$ that indicate the specific parts of F (e.g., In_1 indicates first 0.5% of F ; In_2 indicates first 2.5% of F), where n is the total number of indexes. Furthermore, u_1 calculates the hash codes of partial F based on the indexes as $HC(M) = \{HC(M_1), HC(M_2), \dots, HC(M_N)\}$. Then u_1 sends the data package $dp_1 = \{u_{1,id}, REK_{u_1}, CT\}$ to Pub-CSP and sends the data package $dp_2 = \{F_{id}, u_{1,id}, IN, HC(M)\}$ to Pri-CSP. Last but not least, Pub-CSP and Pri-CSP both maintain the ownership list separately for each data, what saved in Pub-CSP is $P_1 = \{(F_{id}, CT, H(M), (u_{1,id}, REK_{u_1}))\}$, and what saved in Pri-CSP is $P_2 = \{(F_{id}, IN', HC'(M), EK, (u_{1,id}, rk_{u_0 \rightarrow u_1}))\}$,

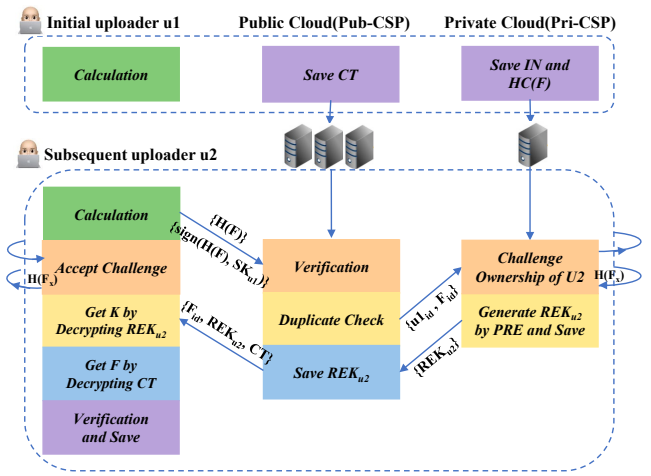


Fig. 3. Procedure of deduplication.

where IN' and $HC'(M)$ is randomly selected from IN and $HC(M)$.

Step 4 - Duplicated Data Upload: Later on, DU u_2 wants to store the same data F at Pub-CSP by sending the data package $dp = \{H(F), sign(H(F), SK_{u_2})\}$.

Step 5 - Deduplication: Pub-CSP performs duplication check as in step 2 after verifying that $u_{2,id}$ is not in the ownership list. When the duplication check is positive, Pub-CSP contacts Pri-CSP for deduplication. Pri-CSP further verifies the ownership of DU u_2 by challenging the hash code set of F before performing deduplication, which ensures the data ownership, since there is a probability that $H(F)$ is eavesdropped or gained by the malicious party. If the ownership verification is positive, Pri-CSP generates the re-encrypted key REK_{u_2} and sends it back to Pub-CSP for saving. Once Pub-CSP receives REK_{u_2} , it sends REK_{u_2} with ciphertext CT to u_2 . When u_2 gets REK_{u_2} , he/she decrypts it with sk_{u_2} to get K . Then u_2 gets plaintext of data F by running $Decrypt(K, CT)$ and checks data consistency, while if u_2 does not want to check data consistency immediately, he/she can check it by downloading CT and $H(F)$ later. Through data deduplication, both u_1 and u_2 can access the same data F that is stored only once at Pub-CSP.

C. Ownership Revocation

When users who have valid ownerships request Pub-CSP to delete or modify their data in the cloud storage, they should be revoked from the ownership list and deterred from accessing the data after data deletion or modification to provide forward secrecy.

Ownership Revocation of Data Holder. Figure 4 shows the procedure of data update by DU u_2 in the context of data deduplication.

Step 1: User u_2 sends a request of data deletion to Pub-CSP by providing $dp = \{F_{id}, sign(F_{id}, SK_{u_2})\}$.

Step 2: Pub-CSP deletes the storage record of u_2 after verifying the signature. Then Pub-CSP asks owner u_1 /owner

TABLE III
COMMUNICATION OVERHEAD.

Scheme	For initial uploader				For subsequent uploader
	Upload message size	Download message size	Rekeying message size	Key size	Upload message size
CE	$C_C + C_H + C_{ID}$	C_C	—	C_K	$C_C + C_H + C_{ID}$
RCE	$C_C + C_K + C_H + C_{ID}$	$C_C + C_K + C_H$	—	C_K	$C_C + C_K + C_H + C_{ID}$
DeupDUM	$C_C + C_K + C_H + C_{ID} + C_P$	$C_C + C_K + C_H$	C_P	$C_K + C_P$	$C_C + C_K + C_H + C_{ID} + C_P$
Our scheme	$C_C + C_H + C_{HC} + C_{ID}$	$C_C + C_K + C_H$	C_K	C_K	$C_H + C_{ID}$

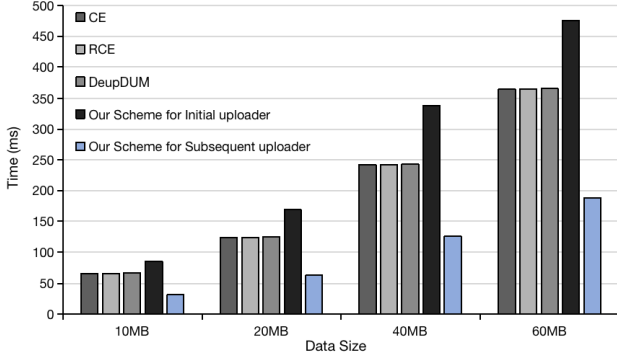


Fig. 5. Computation time for upload.

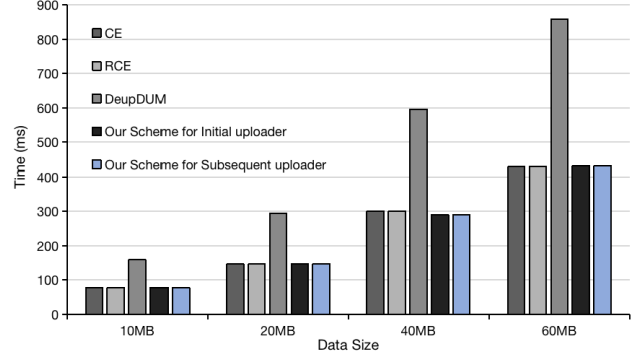


Fig. 6. Computation time for download.

hash code set of data F , C_K denotes the size of a key, C_P denotes the size of a public key.

For the first upload of data F , scheme CE, RCE, and DeupDUM have the same upload message sizes. In our scheme, it increases the size of the hash code set $HC(F)$ used for DU 's ownership verification before deduplication. However, for the subsequent upload of F , our scheme only uploads $H(F)$ before ownership is verified or access is checked while the other schemes need to re-upload all messages every time as shown in Table 3.

Concerning the rekeying message size, scheme CE and scheme RCE do not consider key updating, the DeupDUM and our scheme increase the size of the re-encryption key. However, even though the group key is used to manage ownership revocation, the encryption key K determined by data F still never changes in DeupDUM once it is settled, which is unsecured since the withdrawn owners can collude with Pri-CSP for getting ciphertext before rekeying. Our scheme updated K as long as there is ownership revocation, which is more secure and computation overhead is accepted since it can be performed by Pri-CSP instead of DU.

C. Performance Evaluation

In this section, we compare our scheme with previous works. We follow common practice [33], [36], [40] for fair comparison, each cryptographic operation is implemented by using the umbral library ver. 0.3.0 and Crypto library ver. 1.4.1. We test the time cost of data encryption and decryption algorithm with AES where the key is 128-bit. The data size ranges from 10MB to 60MB. We set $C_K = 128\text{bits}$, $C_T = 128\text{bits}$. The testing environment is Intel(R) Core(TM) i5-7300HQ CPU 3.1 GHz 16.0GB RAM.

1) Computation time for upload:

We tested the efficiency of the file uploading process under different schemes, and scheme DeupDUM requires the same computations as scheme CE and scheme RCE. As shown in **Figure 5**, there is a slight increase in our scheme compared with others since the process in our scheme includes calculating hash code and hash code set of data F , signing and verifying the signature, decrypting re-encrypted key, and encrypting data F with AES. Moreover, We can see that our scheme has great advantages in the duplicated file uploading process. Since our scheme only uploads $H(F)$ before ownership is verified or access is checked while the other schemes need to re-upload all message every time. The abundant communication cost can be saved in our scheme.

2) Computation time for download:

Compared with other schemes, we apply the ownership verification mechanism to verify whether DU has access to whole data or not by challenging DU with random $H(F_x)$ in the hash code set, which can save abundant communication cost when DU does not have ownership. The measure results of computation time for downloading ciphertext between our scheme and other schemes are depicted in **Figure 6**.

3) Computation time for encryption and decryption:

In the encryption stage, the DeupDUM and our scheme resolve the dynamic ownership management problem by performing re-encryption. While our schemes take a shorter time than DeupDUM since Pub-CSP needs to decrypt and re-encrypt the ciphertext for each subsequent uploader in DeupDUM, which will result in high computation complexity as the number of holders grows. The detailed encryption and decryption time for different data sizes (ranging from 10MB to 60MB) is shown in **Figure 7**.

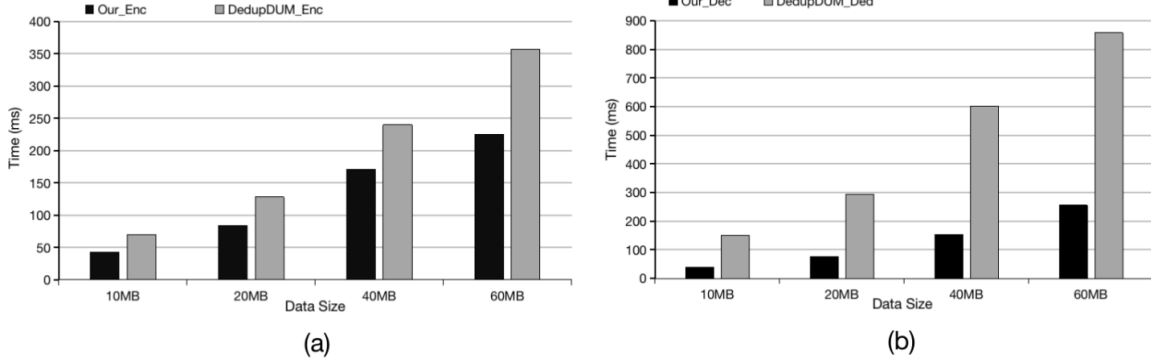


Fig. 7. Computation time for (a) encryption and (b) decryption.

4) Computation time for deduplication:

In this process, the total number of DU is set up as 25, and each of them have a different 2MB-sized file, which means the total size of data saved in the cloud is 50MB when the duplicate ratio(DR) is 0. Specifically, when DR is 20%, which means 5 DUs own the same file, and the rest 20 DUs own different files. From **figure 8**, it shows how computation time changes during each process with DR grows from 0 to 100%. It can be observed that such a process is very efficient, taking less than 0.373 seconds for data upload, and 0.251 for deduplication when the ratio is 100%. Therefore the proposed deduplication scheme can greatly save data uploading time for duplicated data storage in the cloud.

VII. SECURITY ANALYSIS

In this section, we analyze our scheme security involving five security requirements: data privacy, data consistency, data ownership verification, ownership revocation, and collusion resistance. We assume that the underlying basic tools are secure, which include PRE encryption scheme and symmetric encryption scheme. The security of our scheme is ensured by those assumptions.

A. Data Privacy

Since Pub-CSP is not fully trusted even if honest. Plaintext should be protected from the Pub-CSP and unauthorized users. One attack may be launched by the unauthorized cloud user. Supposing an unauthorized user u_2 requests data F with $(F_{id}, u_{1,id})$ (u_1 is valid, and using $u_{2,id}$ will simply not pass the ownership check), Pub-CSP searches the ownership list $P2 = \{F_{id}, CT, H(M), (u_{1,id}, REK_{u_1})\}$, and return $\{F_{id}, CT, REK_{u_1}\}$ to user u_2 based on $(F_{id}, u_{1,id})$. Since REK_{u_1} can only be decrypted by the private key of user u_1 , it is computationally infeasible for user u_2 to obtain plaintext of F by decrypting CT with REK_{u_1} . Another attack scenario can be launched by Pub-CSP. What saved on Pub-CSP is the authorized DU's re-encrypted key that is encrypted through PRE by Pri-CSP and only can be decrypted by DU's private key, since Pri-CSP and authorized users will not collude with Pub-CSP considering their personal profits, it is impossible for the Pub-CSP to get plaintext by cipherkey.

Therefore, data privacy against the honest-but-curious Pub-CSP and unauthorized users is guaranteed.

B. Data Consistency

In the data deduplication scheme, data integrity may be threatened by a poison attack on tag consistency, which could be identified during the decryption process by the data holders. Assuming that attacker u_2 owns the same data F_A with authorized user u_3 , u_2 uses $F_B \neq F_A$ to generate a fake ciphertext CT_B , then uploads $(H(F_A), CT_B)$ to Pub-CSP to pretend to be CT_A . When the authorized user u_3 wants to upload F_A , u_3 sends $H(F_A)$ to Pub-CSP to check duplication. Since $H(F_A)$ exists, Pub-CSP asks Pri-CSP performing deduplication. Pub-CSP sends (CT_B, REK_{u_3}) back to user u_3 after deduplication, and u_3 checks whether $H(Decrypt(De(sk_{u_3}, REK_{u_3}), CT_B)) = H(F_A)$ holds or not, if this is not consistent then u_3 drops the messages and reports this to Pub-CSP. Therefore, our scheme guarantees the data integrity.

C. Data Ownership Verification

In our scheme, the data ownership is verified by challenging DU with random $H(F_x)$ in hash code set (randomly selected specific parts of the data are calculated with their indexes, e.g., the hash code of the first 0.7% of F , the hash code of 10-14% of F). F_x is randomly selected and function $H(*)$ is non-invertible, therefore it is impossible to calculate $H(F_x)$ without the original plaintext.

D. Ownership Revocation

The ownership withdrawn data user should be restricted to access the data. Our scheme guarantees ownership revocation by using Pri-CSP playing a role as owner u_0 . Whenever the data owner u_1 withdraws his/her ownership or some other holders may request to delete or modify their data, the owner u_0 deletes the ownership information of the requestor from the ownership list, and u_0 (if u_1 is offline or revokes his/her ownership or asking u_0 update for him/her) re-encrypts the plaintext with the new symmetric key before re-uploading it to Pub-CSP, and updates the re-encrypted keys of the rest users. Hence, the withdrawn data owner will not be able to pass the

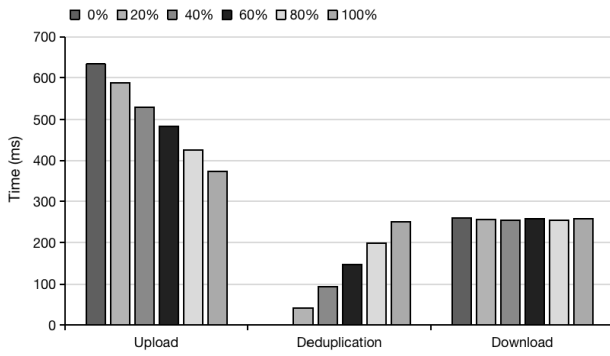


Fig. 8. Computation time for each process with a duplicate ratio grows.

access check and decrypt the latest ciphertext with the non-updated cipher-key.

E. Collusion Resistance

Since Pri-CSP is fully trusted, We further discuss the collusion attacks launched by dishonest Pub-CSP and attackers. First, if unauthorized user u_1 colludes with dishonest Pub-CSP for getting plaintext of data F , Pub-CSP will ask Pri-CSP performing deduplication for u_1 with faking information. Pri-CSP will verify u_1 's ownership of data F before sharing the re-encrypted key REK_{u_1} of F with u_1 . Since u_1 does not have plaintext, he/she can not pass the ownership check to get cipher-key even though Pub-CSP sends u_1 the ciphertext dishonestly. Second, since each cipher-key can only be decrypted by the corresponding user, the unauthorized users are not able to decrypt them even if they collide with each other. Therefore, our scheme guarantees collusion resistance.

VIII. CONCLUSION

Dynamic ownership management is an important and challenging issue in secure deduplication over encrypted data in cloud storage. In this paper, we proposed a practical scheme that managed the encrypted big data with deduplication, based on ownership challenge under a hybrid cloud architecture, where Pub-CSP manages the storage and Pri-CSP plays a role as owner u_0 and proxy at the same time to perform deduplication and dynamic ownership management. Further, our scheme proves that the owner holds the real data alone pass the data ownership, and encrypted data can be securely accessed because only authorized data holders can obtain the symmetric keys used for data decryption. Security analysis, comparison with existing work, and implementation-based performance evaluation showed that our scheme is secure, efficient, and resists collusion attacks and duplicate faking attacks. Therefore, our scheme can efficiently prevent unauthorized users from accessing and downloading the sensitive data of valid users.

REFERENCES

[1] Ibrahim, Abaker, Targio, Hashem, Ibrar, Yaqoob, Nor, Badrul, Anuar, and Salimah, "The rise of "big data" on cloud computing: Review and open research issues," *Information Systems*, vol. 47, no. Jan., pp. 98–115, 2015.

[2] F. M. Awaysheh, M. N. Aladwan, S. Alawadi, J. C. Cabaleiro, and T. F. Pena, "Security by design for big data frameworks over cloud computing," *IEEE Transactions on Engineering Management*, vol. PP, no. 99, 2021.

[3] Duan and Qiang, "Cloud service performance evaluation: status, challenges, and opportunities-a survey from the system modeling perspective," *Digital Communications & Networks*, pp. 101–111, 2016.

[4] Z. Yan, L. Zhang, W. Ding, and Q. Zheng, "Heterogeneous data storage management with deduplication in cloud computing," *IEEE Transactions on Big Data*, pp. 1–1, 2017.

[5] S. Quinlan and S. Dorward, "Venti: A new approach to archival storage," *proc.usenix conf.on file & storage tech*, 2002.

[6] G. R. Blakley and C. Meadows, "Security of ramp schemes," in *Advances in Cryptology, Crypto 84, Santa Barbara, California, Usa, August, 1984*.

[7] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Dupless: Server-aided encryption for deduplicated storage," in *Usenix Conference on Security*, 2013.

[8] X. Jin, L. Wei, M. Yu, N. Yu, and J. Sun, "Anonymous deduplication of encrypted data with proof of ownership in cloud storage," *IEEE/CIC International Conference on Communications in China*, 2013.

[9] J. Li, X. Chen, M. Li, J. Li, P. P. Lee, and W. Lou, "Secure deduplication with efficient and reliable convergent key management," *IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS*, 2014.

[10] J. Li, Y. K. Li, X. Chen, P. Lee, and W. Lou, "A hybrid cloud approach for secure authorized deduplication," *Parallel & Distributed Systems IEEE Transactions on*, vol. 26, no. 5, pp. 1206–1216, 2015.

[11] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Interactive message-locked encryption and secure deduplication," in *Springer, Berlin, Heidelberg*, 2015.

[12] H. Cui, R. H. Deng, Y. Li, and G. Wu, "Attribute-based storage supporting secure deduplication of encrypted data in cloud," *IEEE Transactions on Big Data*, pp. 1–1, 2017.

[13] W. Shen, Y. Su, and R. Hao, "Lightweight cloud storage auditing with deduplication supporting strong privacy protection," *IEEE Access*, vol. 8, pp. 44 359–44 372, 2020.

[14] N. Almrezeq, M. Humayun, A. El-Aziz, and N. Z. Jhanjhi, "An enhanced approach to improve the security and performance for deduplication," *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, vol. 12, no. 6, pp. 2866–2882, 2021.

[15] Hur, Junbeom, Koo, Dongyoung, Shin, Youngjoo, Kang, and Kyungtae., "Secure data deduplication with dynamic ownership management in cloud storage," *IEEE Transactions on Knowledge & Data Engineering*, vol. 28, no. 11, pp. 3113–3125, 2016.

[16] F. Liu, X. Ren, Y. Liu, K. Lei, and X. Sun, "Exploring and distilling cross-modal information for image captioning," in *IJCAI*, 2019.

[17] F. Liu, X. Wu, S. Ge, X. Zhang, W. Fan, and Y. Zou, "Bridging the gap between vision and language domains for improved image captioning," in *ACM Multimedia*, 2020, pp. 4153–4161.

[18] F. Liu, M. Gao, T. Zhang, and Y. Zou, "Exploring semantic relationships for image captioning without parallel data," in *ICDM*, 2019, pp. 439–448.

[19] F. Liu, X. Wu, C. You, S. Ge, Y. Zou, and X. Sun, "Aligning source visual and target language domains for unpaired video captioning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.

[20] F. Liu, C. You, X. Wu, S. Ge, S. Wang, and X. Sun, "Auto-encoding knowledge graph for unsupervised medical report generation," in *NeurIPS*, 2021, pp. 16 266–16 279.

[21] F. Liu, X. Ren, Z. Zhang, X. Sun, and Y. Zou, "Rethinking skip connection with layer normalization," in *COLING*, 2020, pp. 3586–3598.

[22] F. Liu, X. Ren, X. Wu, S. Ge, W. Fan, Y. Zou, and X. Sun, "Prophet attention: Predicting attention with future attention," in *NeurIPS*, 2020.

[23] F. Liu, Y. Liu, X. Ren, X. He, and X. Sun, "Aligning visual regions and textual concepts for semantic-grounded image representations," in *NeurIPS*, 2019.

[24] F. Liu, X. Ren, Y. Liu, H. Wang, and X. Sun, "simnet: Stepwise image-topic merging network for generating detailed and comprehensive image captions," in *EMNLP*, 2018.

[25] F. Liu, X. Wu, S. Ge, W. Fan, and Y. Zou, "Federated learning for vision-and-language grounding problems," in *AAAI*, 2020.

[26] F. Liu, X. Ren, X. Wu, B. Yang, S. Ge, and X. Sun, "O2NA: an object-oriented non-autoregressive approach for controllable video captioning," in *ACL/IJCNLP (Findings)*, 2021.

- [27] F. Liu, C. Yin, X. Wu, S. Ge, P. Zhang, and X. Sun, "Contrastive attention for automatic chest x-ray report generation," in *ACL/IJCNLP (Findings)*, 2021.
- [28] F. Liu, X. Wu, S. Ge, W. Fan, and Y. Zou, "Exploring and distilling posterior and prior knowledge for radiology report generation," in *CVPR*, 2021.
- [29] F. Liu, S. Ge, and X. Wu, "Competence-based multimodal curriculum learning for medical report generation," in *ACL/IJCNLP*, 2021.
- [30] F. Liu, X. Ren, G. Zhao, and X. Sun, "Layer-wise cross-view decoding for sequence-to-sequence learning," *CoRR*, vol. abs/2005.08081, 2020.
- [31] F. Liu, M. Gao, Y. Liu, and K. Lei, "Self-adaptive scaling for learnable residual structure," in *CoNLL*, 2019, pp. 862–870.
- [32] W. Mi, K. Ota, L. He, J. Lei, C. Gu, and S. Zhou, "Secure data deduplication with reliable key management for dynamic updates in cps," *IEEE Transactions on Computational Social Systems*, vol. 2, no. 4, pp. 137–147, 2016.
- [33] H. Yuan, X. Chen, T. Jiang, X. Zhang, Z. Yan, and Y. Xiang, "Dedupdum: Secure and scalable data deduplication with dynamic user management," *Information Sciences*, vol. 456, pp. 159–173, 2018.
- [34] P. K. Premkamal, S. K. Pasupuleti, A. K. Singh, and A. Pja, "Enhanced attribute based access control with secure deduplication for big data storage in cloud," *Peer-to-Peer Networking and Applications*, no. 3, 2021.
- [35] S. Zhang, H. Xian, Z. Li, and L. Wang, "Secdedup: Secure encrypted data deduplication with dynamic ownership updating," *IEEE Access*, vol. 8, pp. 186 323–186 334, 2020.
- [36] J. R. Douceur, A. Adya, W. J. Bolosky, P. Simon, and M. Theimer, "Reclaiming space from duplicate files in a serverless distributed file system," in *Distributed Computing Systems, 2002. Proceedings. 22nd International Conference on*, 2002.
- [37] J. Liu, N. Asokan, and B. Pinkas, "Secure deduplication of encrypted data without additional independent servers," in *CCS*. ACM, 2015, pp. 874–885.
- [38] D. V. K. Vengala, D. Kavitha, and A. P. S. Kumar, "Secure data transmission on a distributed cloud server with the help of hmca and data encryption using optimized cp-abe-ecc," *Cluster Computing*, vol. 23, no. 5, 2020.
- [39] M. Blaze, "Divertible protocols and atomic proxy cryptography," *EU-ROCRYPT 1998*, 1998.
- [40] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2013.